

ESP32 Dual-IMU Firmware: Performance Analysis

Overview: In dual-IMU mode the ESP32 reads from a BNO055 and an MPU6050 each 50–100×/s, fuses their orientation quaternions (with sign continuity, smoothing and SLERP), and publishes via micro-ROS. Below we analyze each subsystem and recommend optimizations for <50 ms latency and ≥ 50 Hz rate.

1. I2C Communication Optimization

- **Current structure:** Sensor reads are done sequentially in the timer callback: `bno.getQuat()`, `bno.getVector(VECTOR_GYROSCOPE)`, `bno.getVector(VECTOR_ACCEL)` for the BNO055, then `mpu.getEvent(&accel,&gyro)` for the MPU6050. Each call entails an I²C transaction (start + address + data + stop). On a 400 kHz bus this is relatively fast – e.g. reading 2 bytes takes on the order of 70 μ s ¹ – but multiple reads add up (e.g. 3 reads/sensor ~ 0.2 ms total).
- **Batching reads:** Combine contiguous register reads where possible. For example, if BNO055's quaternion registers are sequential, issue a single multi-byte `Wire.requestFrom()` to read all quaternion fields at once rather than four separate calls. This removes extra start/stop overhead. Similarly, read the MPU6050's accel+gyro registers in one block (the Adafruit library's `getEvent()` often already does this).
- **Dual I²C buses:** The ESP32 has two I²C peripherals. Create two `TwoWire` instances (or use `Wire` and `Wire1`) and put each IMU on its own bus ². For example, assign BNO055 to I2C-A (SDA21/SCL22) and MPU6050 to I2C-B (other pins). This allows true concurrent bus access with no arbitration, effectively halving bus wait time. (E.g. see example: use `TwoWire I2Cone=TwoWire(0); I2Ctwo=TwoWire(1); I2Cone.begin(SDA1,SCL1,400000); I2Ctwo.begin(SDA2,SCL2,400000); bno.begin(addr,&I2Cone); mpu.begin(addr,&I2Ctwo)` ².)
- **Bus contention & hardware:** With both sensors on the same bus, ensure strong pull-ups (e.g. 2.2–4.7 k Ω) and no other masters. If more sensors were added, consider an I²C multiplexer or even SPI. In practice, two devices cause minimal capacitive loading, but use fast-mode (400 kHz) and check BNO055 timing constraints. Confirm `Wire.setClock(400000)` is used.
- **Measured latency:** As a rule of thumb, reading 64 two-byte devices at 400 kbps takes ~ 4.48 ms ¹ (about 0.07 ms per device). Our two-IMU setup needs only a few hundred μ s for I²C per cycle, so bus speed is not the limiting factor – but batching and dual-bus help headroom.

2. Quaternion Processing Efficiency

- **Expensive operations:** The code uses quaternion normalization (`sqrt`), *spherical linear interpolation* (SLERP, with `acos`, `sin`, `cos`), and gyro-based prediction (`sin/cos`). These

trigonometric calls are relatively heavy on a microcontroller ³. For example, the slerp in `fuse_quaternions()` and the tracker's smoothing both use SLERP with full trig.

- **Simplify interpolation:** Where smoothness permits, replace SLERP with normalized linear interpolation (NLERP). Weighted LERP (component-wise blend + one normalization) uses only multiplies/adds+1 sqrt, which is much cheaper ³. For instance, `fuse_quaternions(q1,q2, $\alpha$ )` can do `q = (q1*(1- $\alpha$ ) + q2_aligned* $\alpha$ ).normalize()`. (The BlitzMax docs note "slerp is more computationally expensive than linear interpolation" ³.) Many games use NLERP as an efficient approximation.
- **Disable nonessential smoothing:** The QuaternionTracker applies jump smoothing (slerp towards target) and glitch detection. If sensor outputs are stable, you might set **always-on smoothing $\alpha=0$** or very low, and skip the glitch branch. That eliminates repeated SLERP calls. You can still align hemisphere by simple dot-product sign flips (cheap) but skip full SLERP if not critical.
- **Normalization:** Currently each quaternion is normalized (one sqrt/divide) every update. You might relax this if small drift is tolerable, or normalize only when norm deviates beyond a threshold, saving CPU.
- **Gyro prediction:** The `predictFromGyro` uses `sin/cos`. If the time delta `dt` is fixed/known, you could precompute or approximate small-angle rotation (e.g. use first-order Taylor for very small angles, or skip if $|\omega| \cdot dt \ll 1$). If orientation jumps are rare, consider not performing gyro-prediction every cycle.
- **Benchmark suggestion:** Profile these steps by timing `micros()` around each block (normalize, slerp, etc.) to see hotspots. Reducing SLERP calls or trig solves much load.

3. micro-ROS Performance

- **Publish overhead:** Each `rcl_publish` sends a UDP packet over WiFi and involves serialization. Micro-ROS on ESP32 can have nontrivial latency per message (often several ms on WiFi). Publishing **three** IMU topics per 10 ms tick can easily consume >10 ms overall. (Anecdotally, one user found 3 msg at 20 ms interval took ~100 ms ⁴.) We must streamline messaging.
- **Topic design:** If full raw IMU1 and IMU2 messages aren't needed by the host, consider skipping or combining. For example, only publish the *fused* orientation, and optionally the two raw quaternions as one custom message or smaller subset. Fewer publishers reduces overhead.
- **QoS and history:** Use **best-effort** QoS (UDP mode) instead of reliable, to avoid retransmits. Micro-ROS allows setting `publisher_options.qos` to best-effort – this "improves publication throughput" by not waiting for ACKs. Also set history depth to 1 (keep only latest message) so old data isn't queued. For example, use `rmw_qos_profile_default` with `reliability=Best_Effort`.
- **Executor/timers:** The current single timer does all reads, processing and three publishes. Consider splitting: e.g., one timer/task reads sensors and computes fusion, another (or the main loop) handles

publishing. Or after reading & processing, call `rcl_publish()` directly without an intervening executor spin, minimizing spin overhead. Ensure the executor's spin or spin_some is called frequently but doesn't introduce extra blocking.

- **WiFi settings:** Ensure WiFi is in station mode with strong signal. Use a dedicated agent PC on the same subnet (static IP if needed) to reduce network latency.
- **Benchmark:** Measure actual loop time including `rcl_executor_spin_some()`. If publish is the bottleneck, consider reducing message size: e.g. trim covariance matrices (empty or zero), or pack quaternion in custom msg.

4. Code Structure and FreeRTOS

- **Dual-core tasks:** The ESP32 is dual-core (core 0 and 1) and supports FreeRTOS. By default, `setup()` / `loop()` run on core 1 ⁵. We can create FreeRTOS tasks pinned to cores to parallelize work. For instance, use `xTaskCreatePinnedToCore(taskFunc, "SensorTask", stack, NULL, priority, NULL, 0)` to run sensor acquisition on core 0, and leave micro-ROS publishing on core 1 ⁵. This separates the heavy I2C+fusion work from the ROS comms.
- **Task splitting:** For example:
 - *Task A (Core 0):* In a fast loop (or high-priority timed loop) perform I2C reads and quaternion fusion. Use `vTaskDelayUntil` or a hardware timer to maintain ~10 ms period.
 - *Task B (Core 1):* Runs the micro-ROS executor (e.g. `rcl_executor_spin_some`) and handles `rcl_publish`. It can read the latest fused data from a thread-safe queue or shared variable updated by Task A.
This way, a slow WiFi publish won't block the next sensor read.
- **Timings & delays:** Replace any `delay()` calls inside critical loops with FreeRTOS delays or let the task block (so it yields to the OS). The main loop currently does `delay(1);` after spinning the executor, which is fine as it yields.
- **Memory/dynamic allocation:** The code uses static message structs (no `new`), so heap use is minimal. Ensure the stack sizes for tasks are sufficient (especially if deep math). Avoid using `std::string` or other heap-allocating features in loops.
- **Remove debug I/O:** Serial/printf calls in the timer (under `ENABLE_DIAGNOSTIC_LOGGING`) can introduce latency and jitter. Disable or remove these debug prints for production timing.
- **Citing dual-core:** As noted in tutorials, `xTaskCreatePinnedToCore(..., core=0)` lets you assign code to specific cores ⁵. Use that to run heavy loops on the otherwise-idle core.

5. Madgwick Filter Cost

- **Computation load:** The FastMadgwick filter (`updateIMU`) does ~2 normalizations (sqrt) and a dozen multiplies/adds per update. While surprisingly efficient for its accuracy, it still costs CPU time (~0.1–1 ms per call depending on frequency). On an ESP32 FPU, this is feasible, but profiling is recommended.
- **Tuning β (beta):** The filter gain β trades responsiveness vs. smoothness. A **smaller β** reduces corrections (smoother output, less noise), a **larger β** reacts faster. If your motion is gentle, lowering β can let you run the filter at a slightly lower update rate (or skip some updates). If you see jitter, a smaller β helps.
- **Simplify if needed:** If the application can tolerate less accuracy for speed:
 - Replace Madgwick with the **Mahony** filter, which has a similar concept but uses only a few multiplies (no gradient descent). This often runs faster, though tuning is needed.
 - Or use a simple complementary filter (just low-pass accel + gyro integration) which is trivial to compute (1 α -blend). Madgwick already “fuses” accel+gyro, so in many cases a complementary filter (or Mahony) can achieve adequate results with less math.
 - **Literature note:** Interestingly, Madgwick is *designed* for low-cost IMUs – one survey notes it is “favored for its low computational cost while providing high accuracy” ⁶. However, our use of sin/cos and sqrt means it’s still heavier than a basic complementary filter.

6. Prioritized Implementation Suggestions

1. **Optimize I²C reads:** Batch contiguous reads and use two I2C buses. *E.g.*, create `Wire1` on different pins for MPU6050, `Wire` for BNO055 ². This can cut the I2C portion per cycle by ~50% and reduce occasional bus delays.
2. **Parallelize tasks on dual cores:** Create FreeRTOS tasks pinned to each core. Run IMU reading+fusion on one core and micro-ROS spinning/publishing on the other ⁵. This ensures sensor reads occur on time even if WiFi publish backs up.
3. **Replace SLERP with NLERP:** In fusion and smoothing routines, use a simple weighted average + normalize instead of full SLERP. This avoids costly trig. According to docs, linear interpolation is “faster to compute” and Nlerp yields similar results for small angles ³.
4. **Adjust quaternion tracker parameters:** If safe, set smoothing $\alpha \rightarrow 0$ or remove jump smoothing. Disable glitch checks. This removes repeated SLERPs in `QuaternionTracker`. Only do simple sign-alignment (dot>0 check) instead.
5. **Tune Madgwick β or replace filter:** Experiment with a lower β (smoother output, less frequent heavy updates). If needed, try Mahony or complementary filtering, which use fewer operations.
6. **Optimize micro-ROS:** Use best-effort QoS and minimal message fields. If raw IMU topics are not needed on host, eliminate them. Reduce executor overhead by calling `rcl_publish` immediately

after filling messages and using `rcl_executor_spin_some`, rather than separate callbacks for each message.

7. **Benchmark and refine:** Insert timing (`micros()`) at key points (start/end of I2C reads, filter update, and before/after `rcl_publish`) to profile where the time goes. For example:

```
uint32_t t0 = micros();
read_bno055_quaternion_safe(...);
uint32_t t1 = micros();
madgwick_filter.updateIMU(...);
uint32_t t2 = micros();
// publish...
uint32_t t3 = micros();
Serial.printf("BNO read %.1f ms, filter %.1f ms, publish %.1f ms\n",
              (t1-t0)/1000.0, (t2-t1)/1000.0, (t3-t2)/1000.0);
```

Use these measurements to verify the 10-20 ms budget.

8. **Code sections to adjust:** Key areas include the timer callback in `main.cpp` (where I2C reads and fusion occur) and the `fuse_quaternions` / `QuaternionTracker` methods in `sensor_fusion.h` (for replacing SLERP with NLERP). The FreeRTOS task creation should go in `setup()` (after WiFi and sensor init), splitting out the current `loop()` work.

Summary: The critical bottlenecks are the quaternion fusion math and micro-ROS publish overhead. Targeting those (via lighter interpolation and parallel tasks) yields the largest gains. I²C reads are comparatively minor, but can still be halved by using both I2C controllers. With these changes, keeping ~50 Hz at <50 ms latency should be achievable in dual-IMU mode.

Sources: Hardware I²C timing and ESP32 I2C usage ¹ ² ; quaternion interpolation cost ³ ; ESP32 dual-core tasking ⁵ ; IMU filter performance ⁶ .

¹ microcontroller - Latency of reading from a large array of i2c slaves? - Electrical Engineering Stack Exchange

<https://electronics.stackexchange.com/questions/290795/latency-of-reading-from-a-large-array-of-i2c-slaves>

² ESP32 I2C Communication: Set Pins, Multiple Bus Interfaces and Peripherals | Random Nerd Tutorials

<https://randomnerdtutorials.com/esp32-i2c-communication-arduino-ide/>

³ BRL.Quaternion · BlitzMax

<https://www.blitzmax.org/docs/en/api/brl/brl.quaternion/>

⁴ micro-ros + esp32 so slow : r/ROS

https://www.reddit.com/r/ROS/comments/1em4swt/microros_esp32_so_slow/

⁵ ESP32 Dual Core with Arduino IDE | Random Nerd Tutorials

<https://randomnerdtutorials.com/esp32-dual-core-arduino-ide/>

6 as-proceeding.com

<https://as-proceeding.com/index.php/ijanser/article/download/2028/1969/3823>